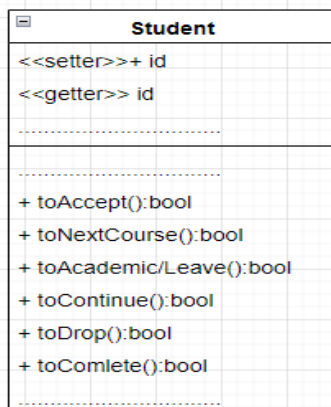


## Принципы SOLID

1. **SOLID:** Single Responsibility Principe (принцип единой ответственности)
2. **SOLID:** Роберт Мартин, книга «Agile Software Development, Principles, and Practices».
3. **Single Responsibility Principe:** принцип единой ответственности:



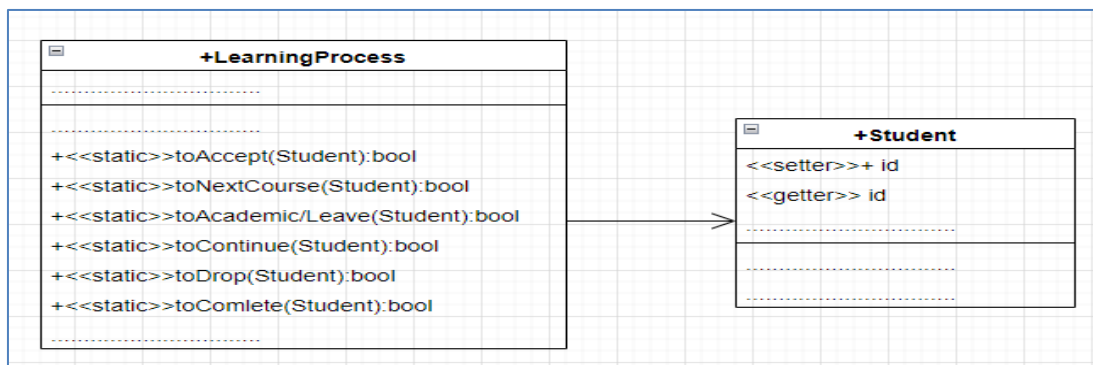
```

public class Student
{
    private int _id;
    public int id {get {return _id;} protected set{_id = value;}}
    public bool toNextCourse()... // на следующий курс
    public bool toAcademicLeave()... // в академический отпуск
    public bool toDrop()... // отчислить
    public bool toContinue()... // восстановить
    public bool toAccept()... // зачислить
    public bool toComplete()... // завершить
}
    
```

```

Student student = new Student();
student.toAccept();
student.toNextCourse();
student.toNextCourse();
student.toAcademicLeave();
    
```

**Ассоциация (использует, взаимодействует, содержит)**



```
public class StudentX
{
    private int _id;
    public int id { get { return _id; }
                  protected set { _id = value; } }
}
```

```
public class LearningProcess
{
    static public bool toNextCourse(StudentX student) ... // на следующий курс
    static public bool toAcademicLeave(StudentX student) ... // в академический отпуск
    static public bool toDrop(StudentX student) ... // отчислить
    static public bool toContinue(StudentX student) ... // восстановить
    static public bool toAccept(StudentX student) ... // зачислить
    static public bool toComplete(StudentX student) ... // завершить
}
```

```
StudentX studentx = new StudentX();
LearningProcess.toAccept(studentx);
LearningProcess.toNextCourse(studentx);
LearningProcess.toDrop(studentx);
```

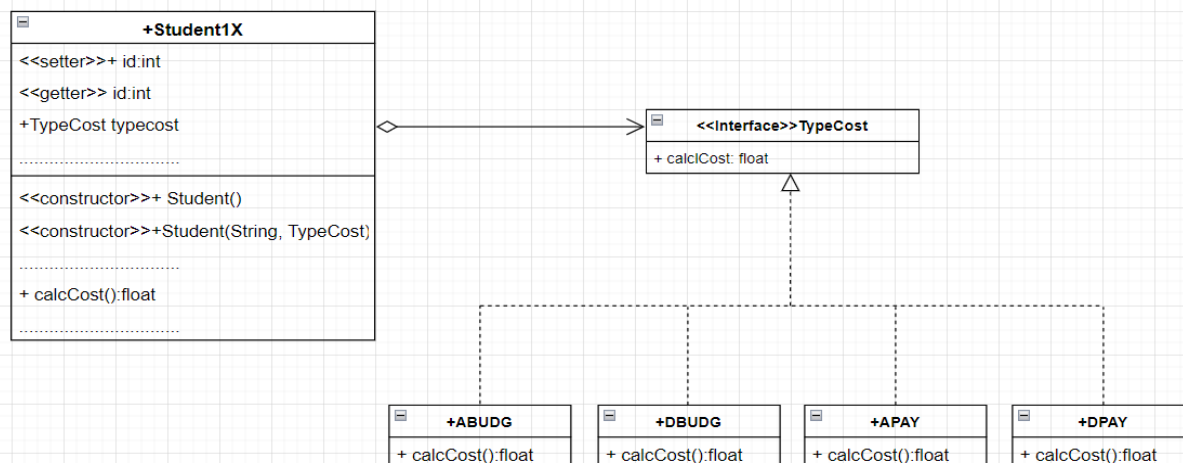
4. **Open/Closed** principle: классы должны быть расширяемы (наследование), но неизменяемы

| +Student1                                 |
|-------------------------------------------|
| <<setter>>+ id                            |
| <<getter>> id                             |
| .....                                     |
| <<constructor>>+ Student()                |
| <<constructor>>+Student(String, TYPECOST) |
| .....                                     |
| + calcCost():float                        |
| .....                                     |

```
public class Student1
{
    public Student1() { }
    public Student1(string Name, TYPECOST typecost)
    {
        this.typecost = typecost;
    }
    private int _id;
    public int id {get {return _id;} protected set{_id = value;}}
    public TYPECOST typecost = TYPECOST.UNKNW;
    public enum TYPECOST
    {
        UNKNW = 0,    // неопределенный
        ABUDG = 1,    // бюджет,    заочно
        DBUDG = 2,    // бюджет,    день
        APAY = 5,     // платно,    заочно
        DPAY = 6,     // платно,    день
    }
    public float calcCost()
    {
        float rc = 0;
        switch( typecost)
        {
            case TYPECOST.ABUDG:    // rc = ;
                break;
            case TYPECOST.DBUDG:    // rc = ;
                break;
            case TYPECOST.APAY:     // rc =
                break;
            case TYPECOST.DPAY:     // rc =
                break;
            default:                //
                break;
        }
        return rc;
    }
}
```

```
Student1 student1 = new Student1("Петров П.П.", Student1.TYPECOST.ABUDG);
student1.calcCost();
```

паттерн «Стратегия», агрегация (включает)



```

public interface ITypeCost
{
    float calcCost();
}

```

```

public class Student1X
{
    private ITypeCost typecost;
    public Student1X() { }
    public Student1X(string Name, ITypeCost typecost)
    {
        this.typecost = typecost;
    }
    private int _id;
    public int id { get { return _id; } protected set { _id = value; } }
    public float calcCost(){return typecost.calcCost();}
}

```

```

public class ABUDG : ITypeCost
{
    public float calcCost()
    {
        float rc = 0;
        // rc =
        return rc;
    }
}

```

```

public class DBUDG : ITypeCost
{
    public float calcCost()
    {
        float rc = 0;
        // rc =
        return rc;
    }
}

```

```

public class APAY : ITypeCost
{
    public float calcCost()
    {
        float rc = 0;
        // rc =
        return rc;
    }
}

```

```

public class DPAY : ITypeCost
{
    public float calcCost()
    {
        float rc = 0;
        // rc =
        return rc;
    }
}

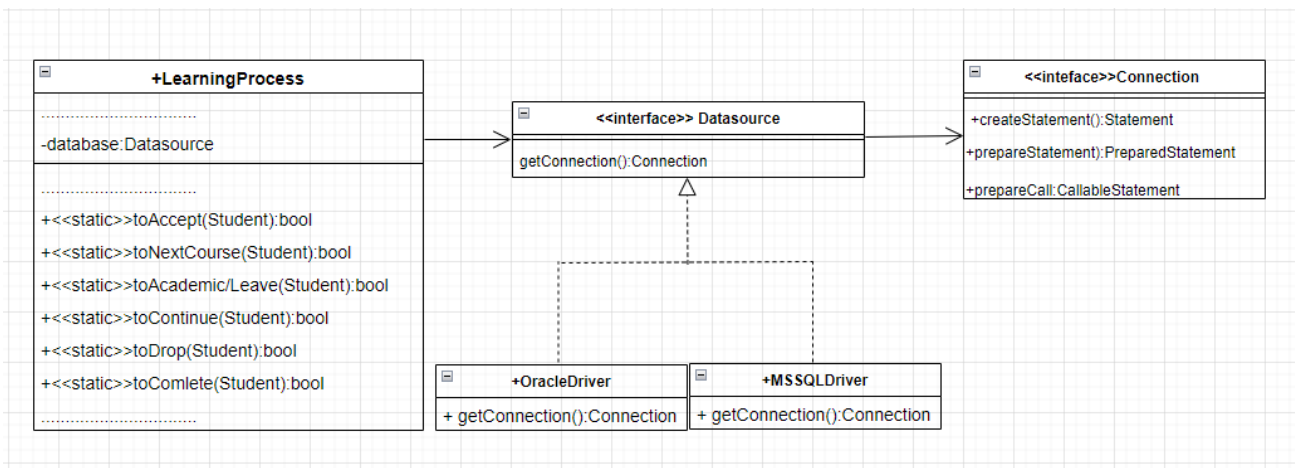
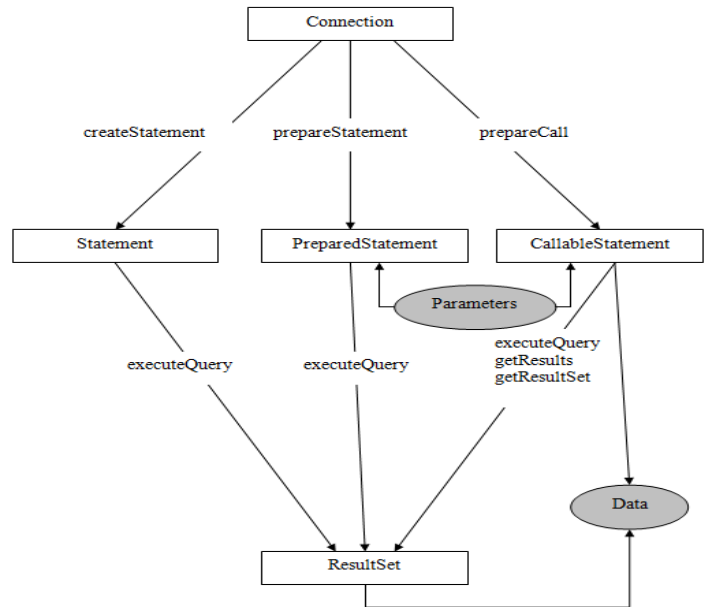
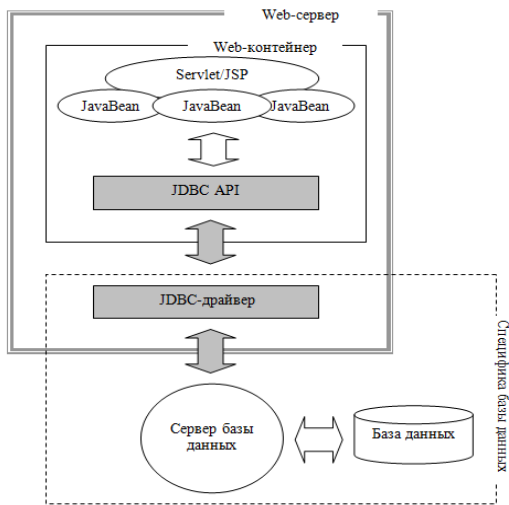
```

```

Student1X student1X = new Student1X("Иванов И.И.", new ABUDG());
student1X.calcCost();

```

5. **Liskov Substitution Principe:** Барбара Лисков (1987) «Data Abstraction and hierarchy»
6. **Liskov Substitution Principe:** производные классы, должны быть разработаны таким образом, чтобы их можно было подставить в клиентский код вместо базового класса и при этом функциональность клиентского кода осталась бы корректной.
- 1) типы параметров одноименных метода производного класса должны иметь такой же тип как у базового или более абстрактный;
  - 2) тип возвращаемого методом значения должен быть такой же как у базового или производным от возвращаемого типа методом базового класса;
  - 3) производный метод не должен выбрасывать исключения не свойственные базовому (можно выбрасывать производные);
  - 4) производный метод не должен ужесточать предусловия;
  - 5) метод не должен ослаблять постусловия;
  - 6) производный класс не изменяет инвариантов (определяющих свойств и методов), в идеале: производный класс только добавляет новые свойства или методы;
  - 7) производный класс не должен изменять приватные поля базового класса (рефлексия).
7. **Interface Segregation:** не должны быть интерфейсы слишком «толстые», интерфейс должен соответствовать принципу «Бритва Оккама» – «не стоит множить сущее без необходимости», можно создавать несколько интерфейсов; не должно быть реализаций интерфейсов с пустой реализацией.
8. **Dependency Inversion Principle:** инверсия зависимости, open/closed principle – с другой стороны, изменение базового класса не должно приводить к изменению производных. Надо зависимость вытаскивать выше по иерархии классов. Низкоуровневые операции – интерфейс, который применяет высокоуровневый класс. Модель: потребитель провайдер. Потребитель формирует требования, провайдер их реализует. Dependency Injection.



9. КОНЕЦ